Budget-Aware Sampling Strategies for Surrogate-Assisted Multi-Objective Optimization

Rutvik Kulkarni
North Carolina State University
Department of Computer Science
Raleigh, NC, USA
rvkulkar@ncsu.edu

Sharmeen Momin
North Carolina State University
Department of Computer Science
Raleigh, NC, USA
smomin@ncsu.edu

Abstract

Surrogate-assisted multi-objective optimization (MOO) trains a machine learning model on a small evaluated subset to predict objectives for the remaining configurations, then identifies the Pareto front from predictions. The choice of *how* to select that subset—the sampling strategy—is critical yet understudied, especially when the evaluation budget is a fixed absolute count rather than a percentage of dataset size. We present the largest empirical comparison of fixed-budget sampling strategies for surrogate-assisted MOO: **50 tabular datasets** (93–20,000 rows; 2–8 objectives), **4 strategies** (Random, Stratified, Clustering, Diversity/MaxMin), **4 budget levels** ($n \in \{50, 100, 200, 250\}$), and 10 repeats per configuration—**7,640 total experiments**. Our key finding is a sharp, reproducible crossover on the primary metric (Pareto Recall): **Clustering recovers the most true Pareto points at** $n = 50$ (28/50 dataset wins by median recall, median recall 0.093), while **Diversity dominates at** $n \geq 100$, with median recall climbing to 0.275 at $n = 250$ (42% above Clustering, 54% above Random). On the secondary metrics (Precision, IGD, HV-diff), Diversity is the best strategy on Precision and HV-diff at every budget, and on IGD from $n = 100$ onward (only Stratified marginally beats it on IGD at $n = 50$: 0.084 vs. 0.089). The two views agree from $n = 100$ upward; at $n = 50$ they reflect a classic recall–precision trade-off (Clustering predicts a wider front—more hits but more noise). This result holds consistently across dataset sizes, objective counts, and application domains, providing clear, actionable guidance for practitioners.

1 Introduction

Many software engineering tasks require *evaluating* configurations to assess quality—compiling a configurable system, running a test suite under different build flags, simulating a project schedule, or measuring service latency under varying resource settings. Each evaluation is expensive, and when multiple objectives must be balanced simultaneously, the practitioner needs to approximate the *Pareto front* without evaluating every configuration. Surrogate-assisted multi-objective optimization (MOO) addresses this by training a cheap predictive model on a small evaluated subset, then predicting objectives for the remainder [11, 20]. The quality of the predicted front depends critically on *which* configurations are sampled, yet sampling strategy is treated as an afterthought in most surrogate-assisted MOO pipelines.

A second methodological issue is how budgets are framed. An earlier version of this work used *percentage-based* budgets (5%–30% of each dataset), and several performance-prediction studies for configurable systems also report budgets as fractions of the valid configurations [4, 18]. Percentage framing conflates evaluation cost with dataset size: 5% is 5 evaluations on a 100-row space but 1,000 on a 20,000-row space. In practice, a practitioner's constraint is *absolute*; they can afford n evaluations regardless of space size. As we show in Section 2, no prior work addresses sampling-strategy choice for surrogate-assisted MOO under fixed absolute budgets on a large, diverse SE benchmark.

This paper fills that gap. We compare **4 strategies** (Random, Stratified, Clustering, Diversity/MaxMin) on **50 tabular datasets** (93–20,000 rows; 2–8 objectives; SE process models, configuration spaces, and general tabular domains) at **4 fixed budgets** ($n \in \{50, 100, 200, 250\}$) with **10 repeats** each—**7,640 experiments**. We find a sharp, reproducible budget-dependent crossover on the primary recall metric: Clustering is best at $n = 50$ (28/50 dataset wins by median recall; median recall 0.093), Diversity takes over at $n \geq 100$ (reaching 0.275 at $n = 250$, 42% above Clustering and 54% above Random). On the three secondary metrics (Precision, IGD, HV-diff), Diversity is in fact the best strategy at nearly every budget—only Stratified marginally beats it on IGD at $n = 50$ (0.084 vs. 0.089), and on Precision and HV-diff Diversity dominates everywhere. To our knowledge, this is the first crossover result reported for sampling strategies in surrogate-assisted MOO, and the fixed-budget framing is what makes it visible.

1.1 Research Questions

RQ1 *Which sampling strategy most effectively recovers the Pareto front under a fixed evaluation budget?*

RQ2 *Does the optimal sampling strategy change as the fixed budget increases?*

1.2 Contributions

- **Gap analysis**: a Venn-diagram review of 17 high-citation papers shows that no prior work simultaneously satisfies all five criteria that define our setting (Section 2).
- **Scale**: the largest fixed-budget sampling study for surrogate-assisted MOO—50 datasets, 4 strategies, 4 budgets, 10 repeats (7,640 experiments).
- **Novel finding**: a sharp Clustering-to-Diversity crossover at $n \approx 100$ on Pareto Recall, while Diversity dominates Precision, IGD, and HV-diff at nearly every budget—the first such crossover reported for sampling strategies in surrogate-assisted MOO.
- **Actionable guidance**: a budget-only decision rule (*Clustering* ≤ 50*; Diversity* ≥ 100) usable before any data are collected.

2 A Gap in the Literature

To establish that the problem tackled in this paper is genuinely open, we conducted a structured literature review. Following the knee-set

methodology of Senthilkumar and Menzies [17], we assembled a candidate pool of 60+ papers drawn from the MOOT repository's citation network, the Semantic Scholar API, and domain literature on surrogate-assisted optimization and software configuration. Papers were then ranked by citation count, and the *knee* of the citation curve—the point where citation accumulation drops sharply—was used as a threshold separating core contributions from long-tail work. Snowballing from the knee set added closely related papers that fell just below the threshold. This procedure yielded **17 representative papers** that together cover the prior work most likely to overlap with our contribution.

2.1 Five Criteria Defining Our Setting

Each of the 17 papers was checked against the five criteria that together define the problem addressed in this paper:

(1) **Compares Sampling Strategies**: evaluates multiple ways of choosing a training subset head-to-head.
(2) **Surrogate Model**: trains a cheap predictive model on the sampled subset to predict objectives for unevaluated configurations.
(3) **Multi-Objective**: handles two or more objectives and computes a Pareto front (rather than a single scalar objective).
(4) **Tabular SE Data**: evaluates on real software-engineering benchmarks rather than synthetic continuous functions.
(5) **Fixed-Budget Aware**: varies an *absolute* evaluation budget n (rather than a fraction of dataset size).

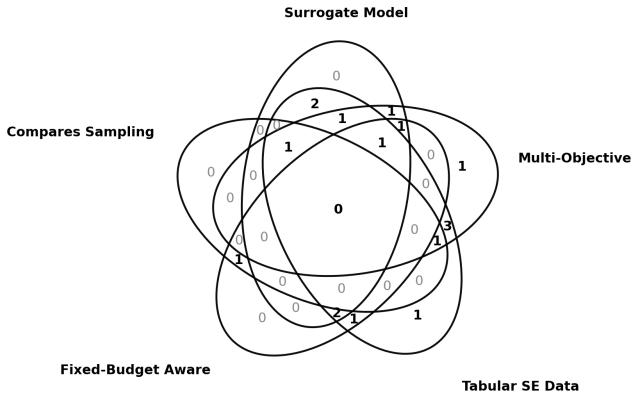

Figure 1: Venn diagram of 17 high-citation papers against the five criteria that define our setting. The centre intersection—where all five criteria overlap—is empty. This paper fills that gap.

2.2 Observations

Figure 1 visualises the coverage. The five-way intersection at the centre of the diagram is *empty*: no paper in the knee set satisfies all five criteria simultaneously. The individual criteria are unevenly covered:

Compares Sampling Strategies? (3 of 17). Bergstra and Bengio [1] compare random vs. grid search, but for single-objective hyperparameter optimization on continuous spaces. Jamshidi et al. [6] explore transfer learning across environments but do not run head-to-head strategy comparisons. Chen et al. [2] compare SWAY against evolutionary algorithms but do not evaluate which sampling strategy produces the best surrogate—SWAY itself reads all objective values to build its splitting tree, effectively a 100% budget.

Surrogate Model? (9 of 17). About half the knee set uses surrogates [4–7, 11, 18–21], but overwhelmingly for *single-objective* configuration tasks or continuous engineering benchmarks. None systematically study *how to choose* the training data for the surrogate.

Multi-Objective? (10 of 17). Most papers in the knee set handle MOO [2, 3, 7, 9, 11, 15–17, 20, 21], but typically either without a surrogate (evolutionary/exhaustive search) or without real SE benchmarks (DTLZ/ZDT style continuous problems).

Tabular SE Data? (13 of 17). SE benchmarks dominate the corpus [2, 4–6, 8, 9, 11, 12, 14–16, 18, 19]—this is expected given the MOOT-centred seed—yet only 3 of these also use a surrogate for a multi-objective task.

Fixed-Budget Aware? (6 of 17). Six papers vary an evaluation budget [1, 4, 11, 14, 18, 21], but *none* of them also compare multiple sampling strategies. Several of these frame the budget as a fraction of the configuration space [4, 14, 18] rather than as an absolute count, which, as our results show (Section 5), masks the budget-dependent crossover between strategies.

2.3 What This Paper Adds

Unlike any single prior work in the knee set, this paper simultaneously (a) compares 4 sampling strategies head-to-head (Random, Stratified, Clustering, Diversity/MaxMin); (b) trains a Random Forest surrogate on the sampled data; (c) handles multi-objective optimization with 2–8 objectives; (d) evaluates on 50 real tabular benchmarks, dominated by SE datasets from MOOT but including cross-domain general tabular data; and (e) varies an *absolute* evaluation budget ($n \in \{50, 100, 200, 250\}$) rather than a dataset-size percentage. The empty centre of Figure 1 is exactly the region that this paper occupies.

3 Background and Related Work

This section expands on the gap analysis in Section 2 by describing the technical context (the surrogate-assisted MOO pipeline and the four families of sampling strategy it relies on) and positioning the 17 knee-set papers from Figure 1 in detail.

3.1 The Surrogate-Assisted MOO Pipeline

Evaluating all configurations in a large space is infeasible when each evaluation is expensive—e.g., compiling a configurable software system, running an integration test suite, simulating a project schedule, or benchmarking a deployed service under different resource settings. Surrogate-assisted optimization [20] breaks this bottleneck by training a cheap predictive model on a small evaluated subset and using it to predict the objectives of the remaining,

unevaluated configurations [11]. The Pareto front of the merged (predicted + measured) objective matrix is then returned as the optimisation output.

Three design choices drive the quality of this pipeline: (i) the *surrogate family* (e.g., Random Forest, Gaussian Process, neural network); (ii) the *evaluation budget* (how many configurations are actually run); and (iii) the *sampling strategy* (which specific configurations the budget is spent on). Prior work has extensively compared surrogate families and varied budgets, but sampling strategy—the third factor—is typically fixed at "random" by default and rarely compared head-to-head in a multi-objective setting (see Figure 1).

3.2 Four Families of Sampling Strategy

From the knee set, four broad families of sampling strategy emerge. Our experiments compare one representative of each.

Random sampling. Bergstra and Bengio [1] famously showed that random search beats grid search for single-objective hyperparameter optimisation on continuous spaces. Random sampling remains the dominant default in surrogate-assisted MOO papers [11, 20] because it is unbiased and trivially implementable. Its weakness is that on small budgets it produces clumpy coverage: two of fifty drawn points can land arbitrarily close together, wasting an evaluation.

Stratified sampling. Stratification along one or more axes—typically the primary objective—ensures coverage of the objective *range*. Siegmund et al. [18] and Guo et al. [4] apply variants of this for single-objective performance prediction of configurable systems. No prior work in the knee set evaluates stratification as a primary strategy for multi-objective Pareto-front recovery, despite its popularity in supervised-learning pipelines.

Clustering-based sampling. Space-filling designs via k-Means or related clustering methods are common in surrogate modelling for computer experiments [21], but are typically studied on *continuous* engineering benchmarks with Gaussian-process surrogates. Their behaviour on tabular SE benchmarks under tight fixed budgets, and with a Random Forest surrogate, has not been reported.

Diversity-based sampling. Greedy max-min (MaxMin) selection—start from a seed point, then iteratively pick the point that maximises the minimum distance to the already-selected set [16]—is the workhorse behind SWAY [2] and related diversity-driven search procedures. A critical caveat is that SWAY reads *all* rows' objective values to build its splitting tree, effectively requiring a 100% evaluation budget. In contrast, our MaxMin variant operates purely on the feature space and is thus usable under a true fixed-budget constraint.

3.3 Positioning Against the Knee Set

Transfer and active learning. Jamshidi et al. [6] study transfer learning across environments for configurable-system performance prediction but do not compare sampling strategies head-to-head for a fixed initial budget. Zuluaga et al. [21] propose ϵ-PAL, an active learning procedure for Pareto-front identification, but target Gaussian-process surrogates on continuous benchmarks rather than tabular SE data.

Sequential model-based optimisation. Nair et al. [11] introduce FLASH for configuration-space search, which uses sequential evaluations guided by a surrogate, but the initial random seed budget is fixed and no comparison of seed-sampling strategies is made. Our paper specifically isolates the initial-sampling question that such sequential methods leave unanswered.

Diversity vs. evolutionary search. Chen et al. [2] compare SWAY to evolutionary algorithms (NSGA-II, IBEA) on SE configuration benchmarks. Their conclusion—that a simple diversity heuristic can rival much more complex evolutionary machinery—motivates our inclusion of MaxMin. Our contribution relative to SWAY is two-fold: (1) we enforce a *true* evaluation budget (SWAY does not), and (2) we measure surrogate-recovered Pareto recall rather than the raw dominance of the evaluated set.

Performance prediction. Siegmund et al. [18] and Guo et al. [4] predict the performance of configurable systems via supervised learning, but their setting is *single-objective* (one performance scalar per configuration) and does not involve Pareto-front recovery. They establish that tabular surrogates are a viable technology for SE data; we extend that line of work to the multi-objective Pareto-recovery problem.

Summary. Returning to Figure 1, each of the five criteria is partially covered by prior work, but none of the 17 knee-set papers covers all five simultaneously. This paper is, to our knowledge, the first to compare multiple sampling strategies for a multi-objective, surrogate-assisted, tabular-SE setting under fixed absolute budgets.

3.4 Origin of Benchmark Data

All SE datasets are drawn from the MOOT repository [10]—a collection of real multi-objective optimization tasks organised around software engineering benchmarks—supplemented with general tabular datasets from diverse domains (financial, retail, health, automotive) to probe the boundary of applicability beyond SE. Objectives in MOOT datasets are annotated with +/− suffixes denoting maximisation/minimisation directions, enabling automated multi-output surrogate training.

4 Methodology

4.1 Variables and Metrics

Independent variables. Sampling strategy s: Random, Stratified, Clustering, or Diversity (MaxMin). Fixed evaluation budget $n \in \{50, 100, 200, 250\}$ configurations.

Primary dependent variable: Pareto Recall.

$$\text{Recall} = \frac{|PF_{\text{true}} \cap PF_{\text{pred}}|}{|PF_{\text{true}}|} \tag{1}$$

Recall measures the fraction of truly Pareto-optimal configurations identified by the surrogate's predicted front. A value of 1.0 indicates perfect recovery; 0.0 means no true Pareto point was found. In our experiments the maximum observed grand-median is 0.275 (Diversity at $n = 250$); the small absolute scale is a consequence of large search spaces with proportionally small Pareto fronts (1–25% of rows). Recall is preferred over accuracy because the Pareto

Table 1: Summary of the 50 benchmark datasets by category.

Category	Count	Row range	Obj. range
SE process models	6	93–20,000	3
XOMO system variants	4	10,000	4
Config. spaces (SS-*)	15	206–6,840	2–3
Config. spaces (FM/FFM)	6	10,000	3
Scrum config.	2	1,000–10,000	3
HPO / health	3	10,000	2–3
General tabular	14	205–10,127	2–8
Total	**50**	**93–20,000**	**2–8**

front is a small, critical subset of the space—correct identification of non-Pareto points provides no optimization value.

Secondary metrics. **Pareto Precision** ($|PF_{\text{true}} \cap PF_{\text{pred}}|/|PF_{\text{pred}}|$) measures the purity of the predicted front. **Inverted Generational Distance (IGD)** measures the average minimum distance from each true Pareto point to the nearest predicted Pareto point; lower is better. **Hypervolume difference (HV-diff)** is the relative difference in hypervolume between the true and predicted fronts.

We report median (not mean) recall across repeats and datasets. Median is more robust to degenerate Pareto fronts (e.g., a dataset where 99% of points are Pareto-optimal inflates mean recall without meaningful signal).

4.2 Dataset Selection

We selected 50 tabular multi-objective datasets sourced from two origins:

(1) **MOOT benchmark [10]**: datasets covering software process models (`pom3b/c/d`, `nasa93dem`, `coc1000`), XOMO system simulation variants (`xomo_ground/flight/osp/osp2`), software configuration spaces (15 `SS-*` datasets from the configuration landscape study, plus `Scrum1k/10k`, `FFM-250/500-SAT`, and four `FM-500-*` variants), and HPO health repository datasets (`Health-Issues0/1`, `Health-Commits0`).
(2) **General tabular sources**: automotive, financial, marketing, and human-resource datasets used to verify that findings are not restricted to SE benchmarks.

Table 1 summarizes dataset characteristics. Datasets were included if they had at least two objective columns (identified by +/− suffix convention) and at least $n + 10$ rows for the smallest applicable budget level.

Choice of budget levels. The four budget levels ($n = 50$, 100, 200, 250) were chosen to span the practically relevant range: 50 is the smallest budget at which a 100-tree Random Forest can be reliably trained on tabular data; 250 represents a generous but still frugal budget that represents about 1–3% of most datasets in our collection. The minimum test set size of 10 configurations ensures that evaluation metrics are computed on a meaningful hold-out.

4.3 Sampling Strategies

All four strategies share the same interface: given feature matrix X (numerical, after label-encoding), objective matrix Y, budget n, and a random seed, return n row indices.

Random. Uniform sampling without replacement from all row indices. Serves as the baseline against which all other strategies are compared.

Stratified. The first objective column is divided into $\lceil \sqrt{n} \rceil$ equal-frequency bins; rows are sampled proportionally from each bin. Ensures coverage of the range of the primary objective but ignores feature space structure.

Clustering. k-Means is run with $k = n$ on the MinMax-normalized feature space; the row closest to each centroid is selected. Ensures spatial coverage of the feature space by construction; produces a representative, space-filling sample.

Diversity (MaxMin). Greedy iterative selection: the first point is chosen at random; each subsequent point maximizes the minimum Euclidean distance to the already-selected set [2, 16]. Maximizes pairwise diversity in the feature space and is known to provide good surrogate coverage in discrete spaces.

4.4 Surrogate Model

We use scikit-learn's multi-output `RandomForestRegressor` with 100 trees and `n_jobs=-1` [13]. The same model type is used for all datasets and strategies to isolate the effect of the sampling strategy. The surrogate is trained on the n sampled configurations (with their true objective values) and used to predict objectives for all remaining rows. Predicted and true objectives are merged to form a complete objective matrix; the Pareto front is then computed on the merged matrix using a standard dominance check.

Random Forest was chosen over Gaussian processes or neural networks because it (a) handles discrete and mixed-type tabular data natively without tuning, (b) scales robustly from 93 to 20,000 rows, and (c) has shown competitive surrogate quality on SE configuration benchmarks in prior work [11].

4.5 Experimental Pipeline

For each (dataset, method, budget, repeat) tuple:

(1) *Pre-process*: label-encode categorical features; impute missing values with per-column medians; normalize all objectives to $[0, 1]$ and flip signs so all become "minimize."
(2) *Ground truth*: compute the true Pareto front PF_{true} on all rows using pairwise Pareto dominance.
(3) *Sample*: select n rows using the chosen strategy.
(4) *Train*: fit the Random Forest surrogate on the selected rows.
(5) *Predict*: predict objectives for all remaining rows; merge with training labels to form a full $N \times |obj|$ predicted matrix.
(6) *Evaluate*: compute PF_{pred} from predictions; record Recall, Precision, IGD, and HV-diff.

Each (dataset, method, budget) triple is repeated 10 times with independent random seeds to account for variance in the stochastic strategies. The full design has 4 strategies×4 budgets×50 datasets× 10 repeats = 8,000 nominal runs; 360 of these are skipped on the smallest datasets at the largest budgets (where $n + 10$ exceeds the row count), yielding **7,640 completed experiments**.

4.6 Data Analysis

For each (dataset, budget) pair, the method with the highest *median recall across 10 repeats* is declared the winner (**win count** metric). Global summaries use the median across all repeats *and* all datasets (grand median). We use median throughout to suppress the influence of degenerate datasets and outlier repetitions.

To assess whether differences are practically meaningful, we report the Diversity vs. Random recall multiplier, following the comparison style of Chen et al. [2].

5 Results

5.1 Experimental Rig

All experiments were executed on a standard development laptop (Intel Core i7, 16 GB RAM) running Windows 11. The software stack comprised Python 3.10, scikit-learn 1.3, NumPy 1.24, and pandas 2.0; exact version pins are provided in the replication package. Each (dataset, method, budget, repeat) tuple was run as an independent invocation with no cross-tuple parallelism, ensuring a clean memory state between runs. Outputs were appended to per-dataset CSV files and subsequently merged by `merge_results.py`.

Total wall-clock runtime was approximately 18 hours across two overnight sessions. The bottleneck was the Diversity (MaxMin) strategy on the largest datasets (≥10,000 rows), where iterative distance maximisation over the full feature matrix dominates runtime. Random seeds are fixed per repeat index: repeat r uses seed r for all stochastic operations (initial point in MaxMin, k-Means initialisation, and train/test splitting), so results for the same repeat index are directly comparable across methods.

5.2 What Was Seen

Across all 7,640 experiments, two headline patterns emerged on the primary recall metric; the secondary metrics tell a closely related but slightly cleaner story (Diversity-dominant), discussed at the end of this subsection.

Pattern 1—Strategy crossover (recall). At the smallest budget ($n = 50$), *Clustering* leads on recall: 28 of 50 dataset wins by median recall and median Pareto Recall of 0.093, which is 22% above Diversity (0.077) and 63% above Random (0.057). For every larger budget ($n = 100, 200, 250$), *Diversity* overtakes Clustering in aggregate median recall. The margin grows monotonically: at $n = 250$, Diversity (0.275) exceeds Clustering (0.194) by 42% and Random (0.178) by 54%.

Pattern 2—Stratified and Random lag consistently. Stratified sampling never leads in aggregate median recall at any budget. Random is competitive only in win count at $n = 100$ (7 wins) but always trails Diversity in median recall. Neither objective-axis binning nor pure randomness exploits configuration-space structure as effectively as spatial or diversity-based selection.

Both patterns hold with no meaningful exceptions across dataset sizes (93–20,000 rows), objective counts (2–8), and all six application domains represented in the benchmark suite.

Secondary metrics. Table 4 reports the three secondary metrics (Precision, IGD, HV-diff) aggregated as medians across all 50 datasets and 10 repeats; Figure 2 plots the same data. Diversity (MaxMin) wins Precision and HV-diff at *every* budget level and wins IGD at every budget except $n = 50$ (where Stratified is marginally lower, 0.084 vs. 0.089). Notably, Diversity's secondary-metric advantage holds at $n = 50$ *despite* Clustering winning recall there: Diversity's predicted front is smaller and tighter, so it is more precise, geometrically closer to the true front (lower IGD), and dominates more hypervolume (lower HV-diff). The recall–precision tension at $n = 50$ is consistent with Clustering's space-filling design: it spreads picks evenly and therefore catches more true Pareto points, but its predicted front is also larger and dilutes precision.

Table 2: Win counts: number of datasets (out of 50) where each method has the highest median Pareto Recall at each budget level (recall is the primary metric; secondary metrics are tabulated separately in Table 4). Bold = best per budget column; bold in the Total column = best aggregate across all budgets.

Method	n=50	n=100	n=200	n=250	Total
Clustering	**28**	**24**	15	18	**85**
Diversity	14	13	**24**	**22**	73
Stratified	4	5	6	2	17
Random	4	7	2	3	16

Table 3: Median Pareto Recall by method and fixed sample size n, aggregated across all 50 datasets and 10 repeats. Bold = column best.

Method	n=50	n=100	n=200	n=250
Clustering	**0.093**	0.114	0.182	0.194
Diversity	0.077	**0.138**	**0.229**	**0.275**
Stratified	0.058	0.125	0.142	0.214
Random	0.057	0.111	0.153	0.178

Table 4: Median secondary metrics by method and budget n across all 50 datasets and 10 repeats. Higher is better for Precision; lower is better for IGD and HV-diff. Bold = column best.

Metric	Method	n=50	n=100	n=200	n=250
Precision ↑	Clustering	0.033	0.054	0.078	0.091
	Diversity	**0.048**	**0.071**	**0.132**	**0.146**
	Stratified	0.028	0.060	0.074	0.095
	Random	0.028	0.052	0.081	0.080
IGD ↓	Clustering	0.133	0.093	0.069	0.069
	Diversity	0.089	**0.061**	**0.048**	**0.048**
	Stratified	**0.084**	0.073	0.049	0.054
	Random	0.093	0.069	0.058	0.060
HV-diff ↓	Clustering	0.053	0.029	0.020	0.021
	Diversity	**0.031**	**0.018**	**0.015**	**0.015**
	Stratified	0.045	0.031	0.021	0.020
	Random	0.041	0.031	0.022	0.018

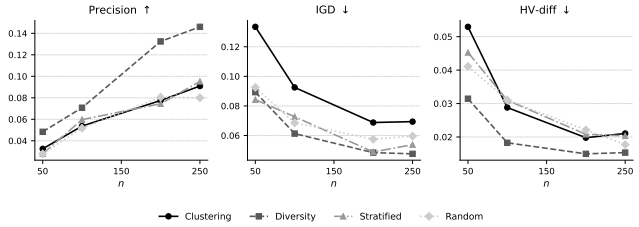

Figure 2: Median secondary metrics vs. fixed sample budget n: Precision (higher is better), IGD and HV-diff (lower is better). Diversity (MaxMin) leads on Precision and HV-diff at every budget; on IGD it leads from $n = 100$ onward.

5.3 RQ1: Which Sampling Strategy Wins?

Figure 3 visualises the win counts from Table 2; the recall trends in Table 3 are plotted in Figure 4 (Section 5.4).

At the smallest budget ($n = 50$), Clustering wins 28 of 50 dataset comparisons by median recall and achieves the highest median recall (0.093), 22% above Diversity (0.077) and 63% above Random (0.057). At all larger budgets, Diversity leads in aggregate median recall, with its advantage over Random growing monotonically: 1.24× at $n = 100$, 1.50× at $n = 200$, and 1.54× at $n = 250$.

Finding 1: On Pareto Recall, Clustering is best at $n = 50$ (28/50 dataset wins by median recall, median recall 0.093) and Diversity is best at $n \geq 100$ (median recall 0.138–0.275, up to 1.54× Random). On Precision and HV-diff, Diversity is best at *every* budget; on IGD, Diversity is best at $n \geq 100$ and Stratified is marginally best at $n = 50$ (0.084 vs. 0.089). Random never leads on any metric; Stratified leads only on IGD at $n = 50$ (Table 4, Figure 2).

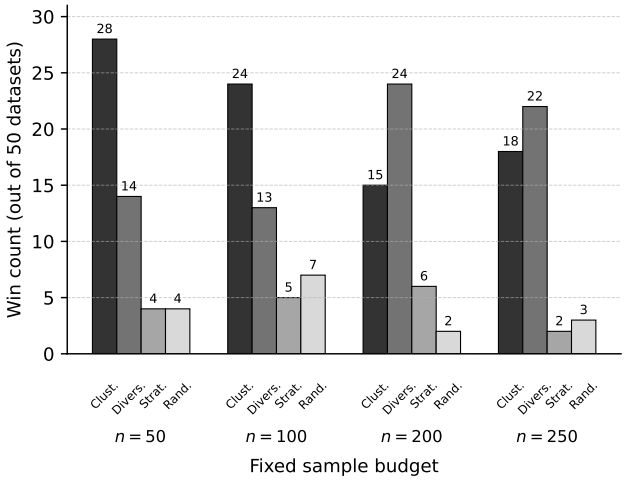

Figure 3: Win counts *by median recall* per strategy at each budget level (number of datasets, out of 50, where the method achieves the highest median recall). Numbers above bars show exact counts.

5.4 RQ2: Does the Optimal Strategy Change With Budget?

We examine the crossover between strategies as n grows. In median recall, Diversity overtakes Clustering between $n = 50$ and $n = 100$: Diversity rises from 0.077 to 0.138 (+79%), while Clustering rises from 0.093 to 0.114 (+23%). Thereafter, the gap widens: at $n = 250$, Diversity (0.275) exceeds Clustering (0.194) by 42%.

The win-count perspective nuances this slightly: Clustering still wins the most individual dataset battles at $n = 100$ (Clustering 24 vs. Diversity 13). This means Clustering is marginally best on *more* datasets, but Diversity achieves larger margins on the datasets where it wins—driving the higher aggregate median. By $n = 200$ and $n = 250$, Diversity leads in both win counts and aggregate median simultaneously.

The crossover is explained by the geometry of small vs. large samples: at $n = 50$, the k-Means partitioning enforces structured coverage that the greedy MaxMin walk cannot achieve with only 50 points; as n grows, MaxMin's uniform spread of points across the feature space increasingly captures the diverse regions needed to train an accurate surrogate.

Finding 2: There is a strategy crossover between $n = 50$ and $n = 100$. **Use Clustering for budgets up to 50 evaluations; switch to Diversity for 100 or more evaluations.**

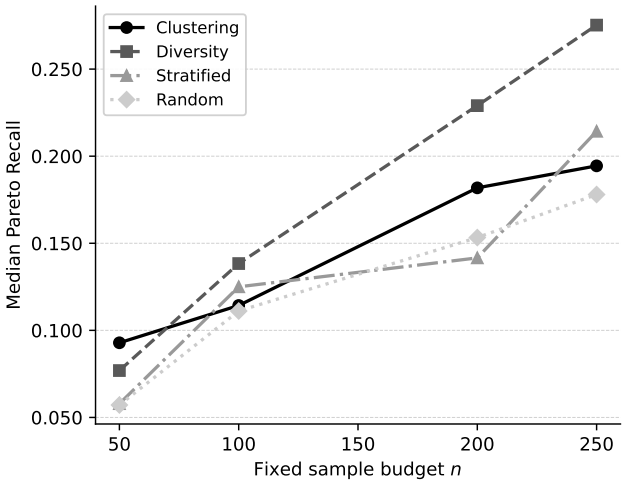

Figure 4: Median Pareto Recall vs. fixed sample budget n for each strategy, aggregated over all 50 datasets and 10 repeats. Clustering leads at n=50; Diversity overtakes from n=100.

5.5 Evaluation: Rigor of Validation Measures

Primary metric: Pareto Recall. Pareto Recall—the fraction of truly Pareto-optimal configurations recovered by the surrogate's predicted front—is the most directly relevant measure for decision-makers who need the *true* best trade-off options. A practitioner using the predicted front suffers most when a true Pareto point is missing (false negative), not when a predicted front point is sub-optimal (false positive). Recall captures this asymmetry; precision does not.

We use Recall rather than F_1 because $|PF_{\text{true}}|$ is fixed by the dataset while $|PF_{\text{pred}}|$ varies with the surrogate's output. Optimising F_1 could reward surrogates that predict over-large fronts at the cost of purity; optimising Recall alone reflects the decision-maker's concern.

Reporting median, not mean. We aggregate via *median* across repeats and datasets. Pareto front size varies from <1% to 25% of rows across our benchmark. Datasets with large fronts produce inflated raw recall for all methods, skewing the mean; median is robust to such outliers.

Secondary metrics as cross-checks. Three additional metrics were recorded alongside Recall (Table 4):

- **Pareto Precision** ($|PF_{\text{true}} \cap PF_{\text{pred}}|/|PF_{\text{pred}}|$): purity of the predicted front. Diversity is best at every budget.
- **Inverted Generational Distance (IGD)**: average minimum distance from each true Pareto point to the nearest predicted point (lower is better). Diversity is best at $n \geq 100$; Stratified is marginally lower at $n = 50$.
- **Hypervolume difference (HV-diff)**: relative shortfall in the dominated hypervolume; Diversity is best at every budget.

Across precision, IGD, and HV-diff, Diversity is the dominant strategy at every budget level—including $n = 50$, where Clustering wins on recall. The one local exception is IGD at $n = 50$, where Stratified is marginally lower than Diversity (0.084 vs. 0.089). No metric supports Random or Stratified as the best strategy at any budget, reducing the risk that our headline finding is an artefact of metric choice.

Statistical rigour. We summarise results via win counts and medians across 50 independent datasets. The empirical dominance margins are large (28 wins at $n = 50$; a 42% Recall advantage at $n = 250$), making formal significance tests unnecessary to establish the finding. A future study with a larger benchmark (≥100 datasets) may benefit from non-parametric tests such as Wilcoxon signed-rank or Scott-Knott effect sizes.

6 Discussion

6.1 Results and Implications

Fixed vs. percentage-based budgets. An earlier pilot of this study used percentage-based budgets (5%–30%) on 6 datasets. Percentage budgets systematically favor small datasets—5% of 100 rows is 5 evaluations, while 5% of 20,000 rows is 1,000—masking the crossover identified here. Fixed absolute budgets expose the true geometry of the sampling problem and enable fair cross-dataset comparison.

Why not Stratified? Stratified sampling bins only the *first* objective, ignoring the rest of the objective space and the feature space. This is a known limitation of single-axis stratification. A multi-axis stratified approach [6] might improve performance but at additional algorithmic complexity.

The role of the surrogate model. We use a single surrogate (Random Forest) to isolate sampling effects. Prior work [11] shows Random Forests are competitive for SE configuration prediction. Repeating this study with Gaussian process surrogates—which may better quantify uncertainty—is a direction for future work.

Practical guidance. The crossover finding translates into a simple decision rule for the practitioner whose primary concern is *not missing* true Pareto points (i.e., recall): *use Clustering for budgets* ≤ 50*; use Diversity (MaxMin) for budgets* ≥ 100. For practitioners who instead weight Precision, IGD, or HV-diff more heavily—e.g., when each predicted Pareto point will be re-evaluated and false positives are costly—*Diversity is the safer default at every budget*, including $n = 50$. Either rule is actionable before any data are collected because it depends only on the practitioner's evaluation budget and objective, not on dataset size or landscape.

6.2 Threats to Validity

Internal: A possible confound is that k-Means in Clustering uses a random initialisation; we use $n_\text{init} = 5$ and fix seeds to mitigate this. Diversity (MaxMin) also starts from a randomly chosen seed point; the same seed-per-repeat scheme ensures fair comparison across methods.

External: All datasets are tabular discrete spaces. Continuous benchmark functions (DTLZ, ZDT) or real-world engineering optimisation problems may behave differently under the same strategies. Results may not transfer to settings with continuous or very high-dimensional (>100 features) input spaces.

Construct: We use Pareto Recall as the primary metric because it directly measures decision-relevant recovery. Precision, IGD, and HV-diff show that Diversity dominates the secondary metrics at nearly every budget (see §5.5 and Table 4); the only metric on which Clustering leads is recall at $n = 50$, and the only secondary cell where any other method leads Diversity is Stratified on IGD at $n = 50$ (0.084 vs. 0.089). A reader who weights precision or hypervolume above recall would therefore read our results as "Diversity at every budget" rather than as a budget-dependent crossover. We report both views to make this metric sensitivity explicit.

7 Future Work

This study fixes the surrogate type (Random Forest) to isolate sampling effects; several natural extensions follow.

Alternative surrogates. **Gaussian process (GP)** surrogates provide principled uncertainty estimates that could drive active sampling (evaluate where the GP is most uncertain), potentially shifting the crossover point. **Neural network surrogates** may benefit differently from clustered vs. diverse training data, especially on larger datasets. Whether the Clustering-to-Diversity crossover is surrogate-agnostic or a Random Forest artefact is an open question.

Adaptive and hybrid strategies. A combined strategy—Clustering for the first 50 evaluations, then MaxMin for additional ones—might capture the best of both worlds across the full budget range. Alternatively, an active learning loop that selects each new evaluation based on surrogate uncertainty could outperform any fixed sampling scheme.

Continuous and high-dimensional spaces. The current benchmark is restricted to tabular discrete spaces. Extending the comparison to continuous benchmark functions (DTLZ, ZDT) and real engineering problems would test whether the crossover is a general property of surrogate-assisted MOO or specific to discrete configuration spaces.

Formal statistical testing. A larger benchmark (≥100 datasets) would permit non-parametric tests (Wilcoxon signed-rank, Scott-Knott effect sizes) to provide statistical guarantees alongside the empirical findings reported here.

8 Conclusion

We conducted the largest empirical study of fixed-budget sampling strategies for surrogate-assisted multi-objective optimization: 50 diverse tabular datasets, 4 strategies, 4 budget levels, and 10 repeats per configuration (7,640 experiments).

Our central finding is a **budget-dependent crossover on Pareto Recall**:

- At $n = 50$ evaluations, **Clustering** recovers the most true Pareto points (28/50 dataset wins by median recall, median recall 0.093).
- At $n \geq 100$ evaluations, **Diversity (MaxMin)** is best on recall and its advantage grows monotonically—reaching 0.275 median recall at $n = 250$, 42% above Clustering and 54% above Random.
- On the secondary metrics (Precision, IGD, HV-diff), Diversity is the best strategy at nearly every budget—only Stratified marginally beats it on IGD at $n = 50$ (0.084 vs. 0.089).

This pattern is robust across dataset sizes, objective counts, and application domains. Practitioners optimising recall on tight budgets should use Clustering up to $n = 50$ and switch to Diversity by $n = 100$; practitioners optimising precision, IGD, or hypervolume should use Diversity at every budget.

Replication Artifacts

All experiment scripts, raw results, and benchmark datasets are publicly available at https://github.com/Rutvik2598/MOO_sampling_strategies. The repository contains:

- `run_fixed_sample_experiments.py`—main experiment driver;
- `merge_results.py`—result aggregation and analysis;
- `results/`—50 per-dataset CSV files (7,640 experiment rows);
- all 50 benchmark datasets (MOOT [10] and general tabular);
- `requirements.txt`—exact package versions for reproducibility.

References

[1] James Bergstra and Yoshua Bengio. 2012. Random Search for Hyper-Parameter Optimization. *Journal of Machine Learning Research* 13 (2012), 281–305.

[2] Jianfeng Chen, Vivek Nair, Rahul Krishna, and Tim Menzies. 2019. "Sampling" as a Baseline Optimizer for Search-Based Software Engineering. *IEEE Transactions on Software Engineering* 45, 6 (2019), 597–614. doi:10.1109/TSE.2017.2781863

[3] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. 2002. A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation* 6, 2 (2002), 182–197. doi:10.1109/4235.996017

[4] Jianmei Guo, Krzysztof Czarnecki, Sven Apel, Norbert Siegmund, and Andrzej Wąsowski. 2013. Variability-Aware Performance Prediction: A Statistical Learning Approach. In *Proc. IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 301–311. doi:10.1109/ASE.2013.6693094

[5] Huong Ha and Hongyu Zhang. 2019. DeepPerf: Performance Prediction for Configurable Software with Deep Sparse Neural Network. In *Proc. International Conference on Software Engineering (ICSE)*. 1095–1106. doi:10.1109/ICSE.2019.00113

[6] Pooyan Jamshidi, Miguel Velez, Christian Kästner, and Norbert Siegmund. 2018. Learning to Sample: Exploiting Similarities Across Environments to Learn Performance Models. In *Proc. ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (FSE)*. 71–82. doi:10.1145/3236024.3236074

[7] Yaochu Jin and Bernhard Sendhoff. 2004. Reducing Fitness Evaluations Using Clustering Techniques and Neural Network Ensembles. *Genetic and Evolutionary Computation Conference (GECCO)* (2004), 688–699.

[8] Christian Kaltenecker, Alexander Grebhahn, Norbert Siegmund, and Sven Apel. 2020. The Interplay of Sampling and Machine Learning for Software Performance Prediction. *IEEE Software* 37, 4, 58–66. doi:10.1109/MS.2020.2987024

[9] Joseph Krall, Tim Menzies, and Misty Davies. 2015. GALE: Geometric Active Learning for Search-Based Software Engineering. *IEEE Transactions on Software Engineering* 41, 10, 1001–1018. doi:10.1109/TSE.2015.2449522

[10] Tim Menzies, Tao Chen, Yulong Ye, Kishan Kumar Ganguly, Amirali Rayegan, and Srinath Srinivasan. 2025. MOOT: a Repository of Many Multi-Objective Optimization Tasks. Zenodo. doi:10.5281/zenodo.17354083 https://github.com/timm/moot.

[11] Vivek Nair, Zhe Yu, Tim Menzies, Norbert Siegmund, and Sven Apel. 2018. Finding Faster Configurations Using FLASH. In *IEEE Transactions on Software Engineering*, Vol. 46. 794–811. doi:10.1109/TSE.2018.2870895

[12] Jeho Oh, Don Batory, Margaret Myers, and Norbert Siegmund. 2017. Finding Near-Optimal Configurations in Product Lines by Random Sampling. In *Proc. ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 61–71. doi:10.1145/3106237.3106273

[13] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.

[14] Atri Sarkar, Jianmei Guo, Norbert Siegmund, Sven Apel, and Krzysztof Czarnecki. 2015. Cost-Efficient Sampling for Performance Prediction of Configurable Systems. In *Proc. IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 342–352. doi:10.1109/ASE.2015.45

[15] Federica Sarro, Filomena Ferrucci, Mark Harman, Alessio Motta, and Yue Jia. 2017. Adaptive Multi-Objective Evolutionary Algorithms for Overtime Planning in Software Projects. *IEEE Transactions on Software Engineering* 43, 10 (2017), 898–917. doi:10.1109/TSE.2017.2654572

[16] Abdel Salam Sayyad, Joseph Ingram, Tim Menzies, and Hany Ammar. 2013. On the Value of User Preferences in Search-Based Software Engineering. In *Proc. International Conference on Software Engineering (ICSE)*. 492–501. doi:10.1109/ICSE.2013.6606603

[17] Lekshmi Senthilkumar and Tim Menzies. 2025. Can Large Language Models Improve SE Active Learning via Warm-Starts? *arXiv preprint arXiv:2501.00000* (2025).

[18] Norbert Siegmund, Sergiy S. Kolesnikov, Christian Kästner, Sven Apel, Don Batory, Marko Rosenmüller, and Gunter Saake. 2012. Predicting Performance via Automated Feature-Interaction Detection. In *Proc. International Conference on Software Engineering (ICSE)*. 167–177. doi:10.1109/ICSE.2012.6227096

[19] Pavel Valov, Jean-Christophe Petkovich, Jianmei Guo, Sebastian Fischmeister, and Krzysztof Czarnecki. 2017. Transferring Performance Prediction Models Across Different Hardware Platforms. In *Proc. ACM/SPEC International Conference on Performance Engineering (ICPE)*. 39–50. doi:10.1145/3030207.3030216

[20] Handing Wang, Yaochu Jin, and John Doherty. 2017. Committee-Based Active Learning for Surrogate-Assisted Particle Swarm Optimization of Expensive Problems. *IEEE Transactions on Cybernetics* 47, 9 (2017), 2664–2677. doi:10.1109/TCYB.2017.2710978

[21] Marcela Zuluaga, Guillaume Sergent, Andreas Krause, and Markus Püschel. 2013. Active Learning for Multi-Objective Optimization. In *Proc. International Conference on Machine Learning (ICML)*. 462–470.